

Checkpoint 5 - MusicSpotlight

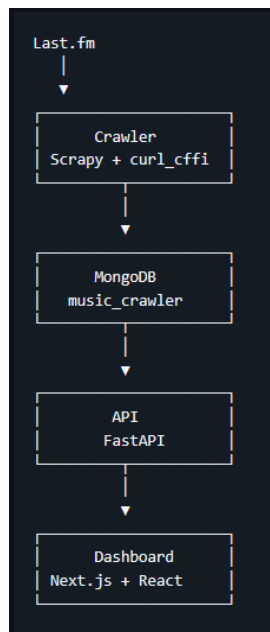
1. Visão Geral do Projeto

O **MusicSpotlight** é um sistema completo desenvolvido para fins acadêmicos e educacionais que integra um Web Crawling, um banco de dados, uma API REST de alta performance e um painel interativo Front-End para a visualização de informações e tendências musicais.

O projeto coleta dados diretamente da plataforma **Last.fm** utilizando spiders desenvolvidos em Scrapy. As informações coletadas são armazenadas e normalizadas em um banco de dados MongoDB, disponibilizadas por meio de uma API construída com FastAPI e apresentadas graficamente aos usuários através de uma interface web moderna baseada em Next.js e React.

2. Arquitetura e Fluxo de Dados

O ecossistema do MusicSpotlight opera de forma modular. O ciclo de vida da informação ocorre sequencialmente por meio das seguintes etapas:



Arquitetura de Componentes

- **Crawler:** Responsável por extrair rankings gerais e dados baseados em tags musicais do Last.fm.
- **Banco de Dados:** Banco de dados não relacional (MongoDB) estruturado com índices

únicos para evitar redundâncias.

- **API REST:** Desenvolvida em FastAPI, fornece rotas estruturadas com serialização baseada em Pydantic.
- **Front-End:** Dashboard interativo em Next.js focado na exibição de gráficos e filtros dinâmicos.

3. Tecnologias Utilizadas

O desenvolvimento do sistema fez uso de uma *stack* tecnológica moderna e orientada a microsserviços e aplicações web reativas:

Back-end e API

- **Python 3.13+:** Linguagem base para o desenvolvimento do crawler e da API.
- **FastAPI:** Framework web moderno e de alto desempenho para construção de APIs REST.
- **Uvicorn:** Servidor ASGI para execução da aplicação FastAPI.
- **Pydantic:** Validação de dados e tipagem em tempo de execução.
- **PyMongo:** Biblioteca oficial de integração Python para o MongoDB.
- **MongoDB:** Sistema de banco de dados NoSQL orientado a documentos.

Crawler e Ingestão de Dados

- **Scrapy:** Framework robusto para web scraping e extração de dados em Python.
- **curl_cffi:** Extensão para simulação avançada de requisições HTTP imitando navegadores reais (essencial para contornar barreiras anti-bot).

Front-End e Visualização

- **Next.js:** Framework React com renderização híbrida.
- **React:** Biblioteca JavaScript para construção de interfaces baseadas em componentes.
- **Tailwind CSS:** Framework utilitário para estilização responsiva.
- **Recharts:** Biblioteca de gráficos customizáveis baseada em React.

5. Módulo Crawler (musicrawling)

Localizado em `back-end/musicrawling/`, este módulo é responsável pela automação de extração de dados do Last.fm. Utiliza o Scrapy em conjunto com o `curl_cffi` para simular requisições legítimas de navegador.

Spider de Rankings (charts)

- **Comando de Execução:** scrapy crawl charts
- **URL Base:** https://www.last.fm/charts
- **Escopo de Coleta:** Músicas mais populares (*Top Tracks*), artistas mais populares (*Top Artists*) e novos lançamentos (*New Releases*).
- **Atributos Coletados:** Posição no ranking, nome do item, artista associado, URL da capa/imagem, URL direta do item, URL do artista e carimbo de data/hora (*timestamp*) da coleta.

Spider de Tags (tags)

- **Comando de Execução:** scrapy crawl tags
- **URL Base:** https://www.last.fm/music
- **Escopo de Coleta:** Categorização por tags musicais. A partir de tags principais e complementares detectadas automaticamente ou injetadas manualmente, o crawler extrai as faixas, artistas e álbuns mais relevantes por gênero/tag.
- **Logs de Execução:** O console reporta métricas de progresso estruturadas como Initial tags, Related tags e Processed tags.

6. Módulo de Banco de Dados

O banco de dados utilizado é o **MongoDB**, estruturado sob a base de dados denominada `music_crawler`.

Coleções Principais

- `tracks`: Armazena as faixas musicais coletadas.
- `artists`: Armazena os dados referentes aos artistas.
- `albums`: Armazena os lançamentos de álbuns mapeados.

Integridade e Tratamento de Dados

Para garantir a consistência das informações, o sistema aplica um índice único baseado na concatenação de `url + chart + tag`. Isso impede a duplicação de entradas idênticas para a mesma categoria de listagem.

O fluxo de inserção passa por pipelines rigorosos:

1. **Validação:** Checagem de integridade dos campos obrigatórios.
2. **Normalização:** Padronização de strings e formatos de data.
3. **Desduplicação em Memória:** Filtro prévio de registros repetidos no ciclo do spider.
4. **Persistência:** Inserção otimizada no MongoDB via PyMongo.

7. Guia de Instalação e Execução

Pré-requisitos do Sistema

- Python 3.13 ou superior configurado no PATH.
- Node.js e gerenciador de pacotes npm instalados.
- Instância do MongoDB rodando localmente na porta padrão (27017).

Configuração do Crawler

```
cd back-end/musiccrawling  
python -m venv .venv
```

```
# Ativação no Windows:  
.venv\Scripts\activate
```

```
# Ativação no Linux/macOS:  
source .venv/bin/activate
```

```
pip install -r crawler_requirements.txt
```

Para popular o banco de dados, execute os spiders:

```
scrapy crawl charts  
scrapy crawl tags
```

Configuração e Inicialização da API

```
cd back-end/api  
python -m venv .venv
```

```
# Ativação no Windows:  
.venv\Scripts\activate
```

```
# Ativação no Linux/macOS:  
source .venv/bin/activate
```

```
pip install -r api_requirements.txt  
uvicorn main:app --reload
```

- **Endpoint Base:** <http://localhost:8000>
- **Documentação Interativa (Swagger UI):** <http://localhost:8000/docs>

Configuração e Execução do Front-End

```
cd front-end/crawler-project
```

```
npm install
```

```
npm install recharts
```

```
npm run dev
```

- **Dashboard Disponível em:** <http://localhost:3000>

8. Especificação da API REST

A API possui rotas modulares desenvolvidas em FastAPI para consulta segmentada dos dados persistidos.

Músicas (/tracks)

- GET /tracks: Retorna as 10 principais músicas do ranking geral.
- GET /tracks/{tag}: Retorna as músicas filtradas por uma tag específica (ex: /tracks/rock).

Artistas (/artists)

- GET /artists: Retorna os 10 artistas mais populares do ranking geral.
- GET /artists/{tag}: Retorna os artistas associados a uma tag específica (ex: /artists/metal).

Álbuns (/albums)

- GET /albums: Retorna os 10 principais álbuns ou lançamentos gerais.
- GET /albums/{tag}: Retorna os álbuns filtrados por tag (ex: /albums/rock).

Consulta Geral (/all)

Permite recuperar o conjunto completo de registros de uma categoria específica por meio de parâmetros de rota:

- GET /all/AL — Retorna todos os Álbuns.
- GET /all/AR — Retorna todos os Artistas.
- GET /all/TR — Retorna todas as Tracks.

9. Modelos de Dados (Esquema JSON)

Track (Música)

```
{
  "rank": 1,
  "name": "Nome da música",
  "artist": "Nome do artista",
  "cover": "URL da capa",
  "url": "URL da música",
  "artist_url": "URL do artista",
  "chart": "top_tracks",
  "time": "2026-10-07T00:00:00",
  "tag": "rock"
}
```

Artist (Artista)

```
{
  "rank": 1,
  "name": "Nome do artista",
  "image": "URL da imagem",
  "listeners": 1000000,
  "url": "URL do artista",
  "chart": "top_artists",
  "time": "2026-10-07T00:00:00",
  "tag": "rock"
}
```

Album (Álbum)

```
{
  "rank": 1,
  "name": "Nome do álbum",
  "artist": "Nome do artista",
  "cover": "URL da capa",
  "listeners": 1000000,
  "url": "URL do álbum",
  "artist_url": "URL do artista",
  "chart": "top_albums",
  "time": "2026-10-07T00:00:00",
  "tag": "rock"
}
```

10. Considerações Éticas e Observações

O projeto foi estruturado estritamente para **fins educacionais e de pesquisa acadêmica**.

As configurações do Scrapy foram planejadas para mitigar impacto na infraestrutura do site alvo:

- Utilização de identificador amigável (USER_AGENT = musiccrawling (educational project)).
- Respeito às diretrizes do arquivo robots.txt.
- Limitação de requisições concorrentes por domínio com adoção de intervalos (*throttling*) entre as requisições.

11. Equipe de Desenvolvimento

- ANDRÉ VICTOR GONÇALVES NASCIMENTO — RM 570567
- DAVI DIAS DE SOUZA FREITAS — RM 574089
- DAVID MIKAEL DIAS DA SILVA — RM 571637
- GABRIEL NOVAGA PEREIRA — RM 573196
- MATHEUS MONTEIRO DA SILVA — RM 573842

12. Referências Bibliográficas

- Last.fm. Disponível em: <https://www.last.fm/>
- Scrapy Documentation. Disponível em: <https://scrapy.org/>.
- FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com/>.
- MongoDB Documentation. Disponível em: <https://www.mongodb.com/>.
- Next.js Documentation. Disponível em: <https://nextjs.org/>.
- React Documentation. Disponível em: <https://react.dev/>.
- Tailwind CSS. Disponível em: <https://tailwindcss.com/>.
- Recharts. Disponível em: <https://recharts.org/>.